

# **CSS5280 Design and Analysis of Complex Systems**

## **Lecture 5-2: Community Detection**

(adapted from prof. Albert-László Barabási's slides)

**Instructor: Zhanzhan Zhao**

School of Humanities and Social Science

School of Artificial Intelligence

School of Data Science (by courtesy)

CUHK, Shenzhen

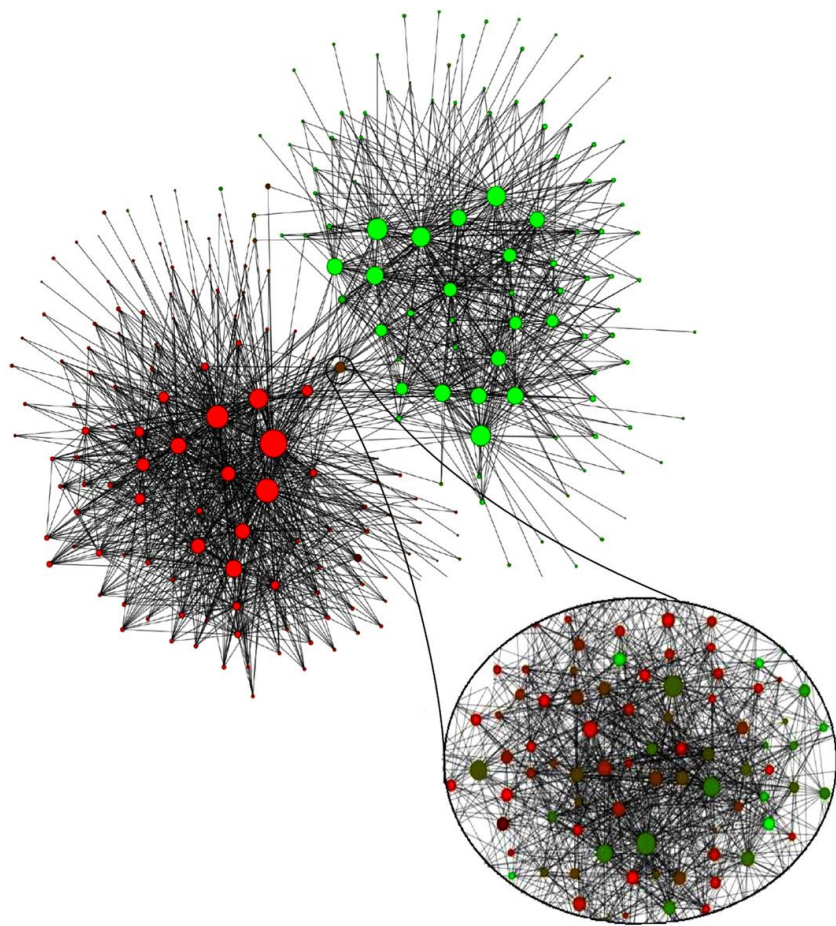
# Introduction





Same area as Massachusetts (~12,000 sq miles)  
Same population as Ohio (~11.5 millions )



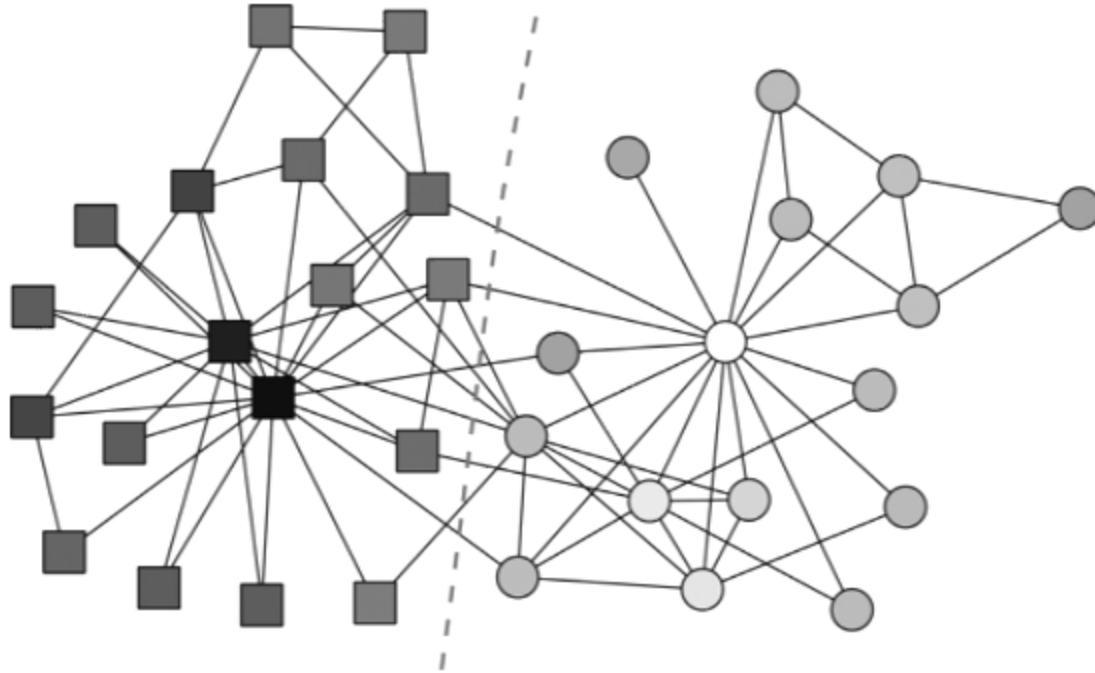


V.D. Blondel et al, *J. Stat. Mech.* P10008 (2008).

A.-L. Barabási, *Network Science: Communities.*

# Examples of communities

# Zachary's Karate Club

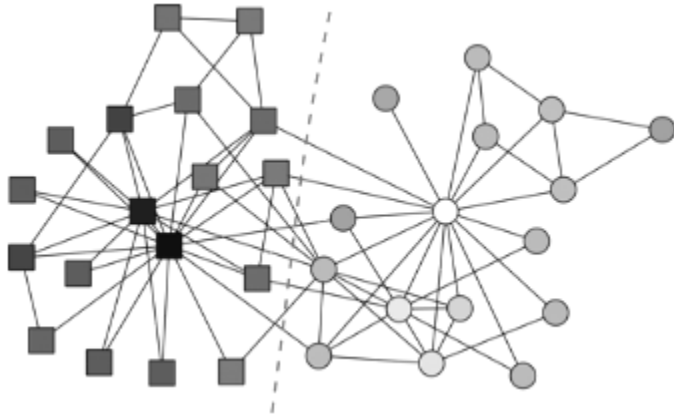


W.W. Zachary, *J. Anthropol. Res.* 33:452-473 (1977).

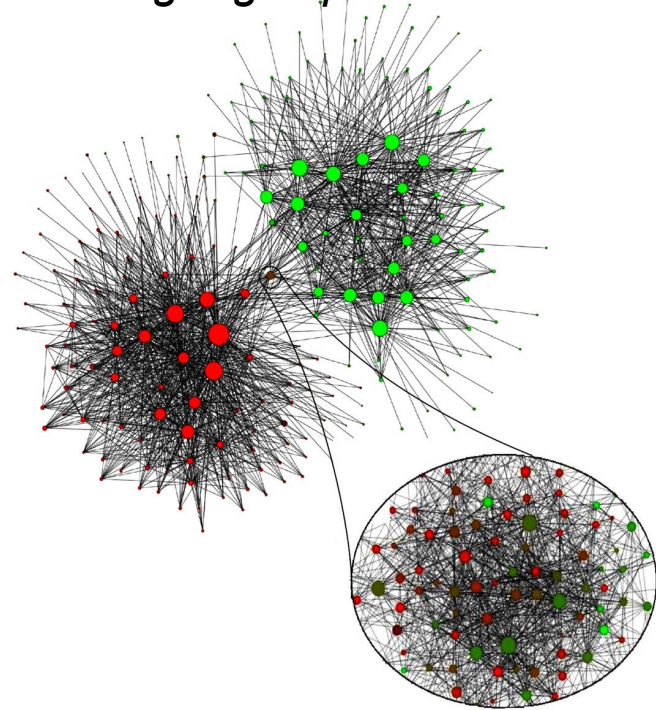
A.-L. Barabási, *Network Science: Communities*.

## Auxiliary information

→ *Karate Club:*  
*Breakup of the club*



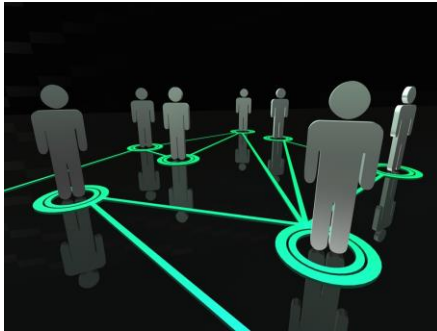
→ *Belgian Phone Data:*  
*Language spoken*





# Basics of communities

We focus on the **mesoscopic** scale of the network



Microscopic



Macroscopic

Mesoscopic

**H1:** A network's community structure is uniquely encoded in its wiring diagram

## H2: Connectedness Hypothesis

A community corresponds to a connected subgraph.

## H3: Density Hypothesis

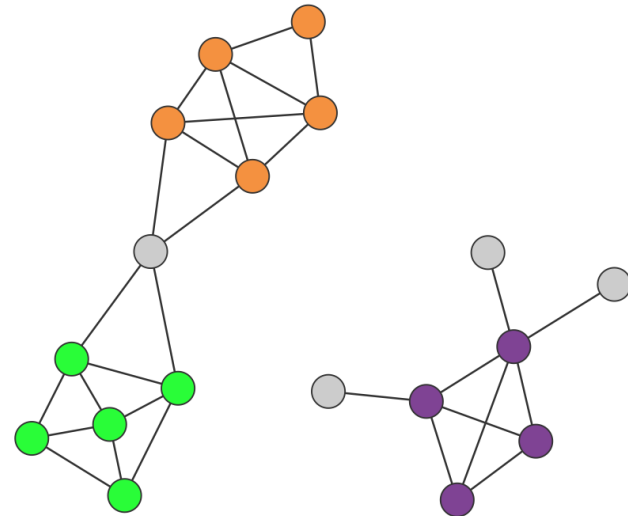
Communities correspond to locally dense neighborhoods of a network.

## H2: Connectedness Hypothesis

A community corresponds to a connected subgraph.

## H3: Density Hypothesis

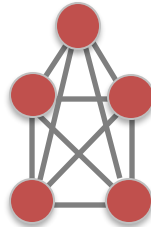
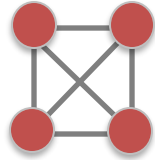
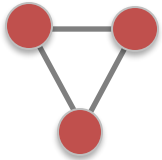
Communities correspond to locally dense neighborhoods of a network.





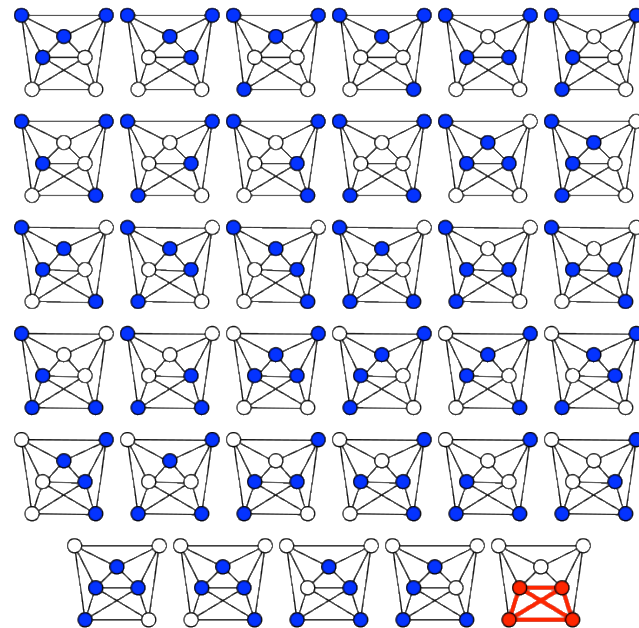
## Cliques as communities

A clique is a complete subgraph of  $k$ -nodes



## Cliques as communities

- Triangles are frequent; larger cliques are rare.
- Communities do not necessarily correspond to complete subgraphs, as many of their nodes do not link directly to each other.
- Finding the cliques of a network is computationally rather demanding, being a so-called NP-complete problem.



How many ways can we partition a network into 2 communities?

## Graph bisection

Divide a network into two equal non-overlapping subgraphs, such that the number of links between the nodes in the two groups is minimized.

Two subgroups of size  $n_1$  and  $n_2$ . Total number of combinations:  $\frac{N!}{n_1!n_2!}$

$$n! \simeq \sqrt{2\pi n} (n/e)^n$$
$$\frac{N!}{n_1!n_2!} \simeq \frac{\sqrt{2\pi N} (N/e)^N}{\sqrt{2\pi n_1} (n_1/e)^{n_1} \sqrt{2\pi n_2} (n_2/e)^{n_2}} \simeq \frac{N^{N+1/2}}{n_1^{n_1+1/2} n_2^{n_2+1/2}}$$

$$n_1 = n_2 = N/2$$

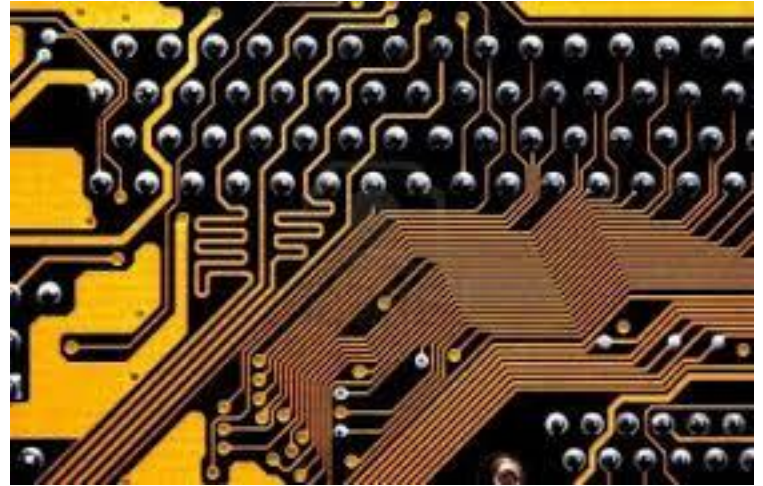
$$\frac{2^{N+1}}{\sqrt{N}} = e^{(N+1)\ln 2/\sqrt{N}}$$

$N=10 \rightarrow 256$  partitions (1 ms)

$N=100 \rightarrow 10^{26}$  partitions ( $10^{21}$  years)

## Graph Partitioning

partition the full wiring diagram of an integrated circuit into smaller subgraphs, so that they minimize the number of connections between them.



2.5 billion transistors

# Number of communities

## Community detection

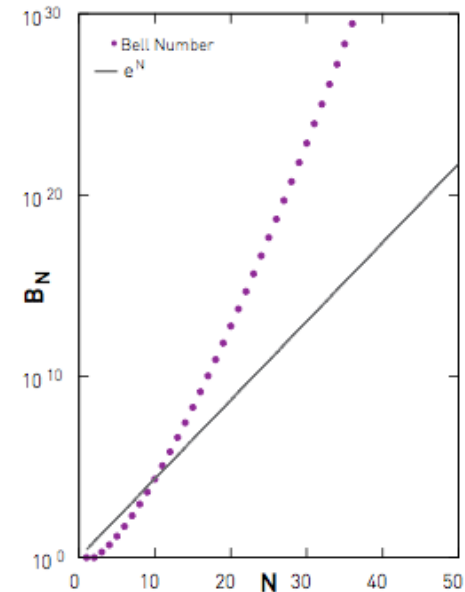
The *number* and *size* of the communities are unknown at the beginning.

## Partition

Division of a network into groups of nodes, so that each node belongs to one group.

$$B_N = \frac{1}{e} \sum_{j=0}^{\infty} \frac{j^N}{j!} .$$

Bell Number: number of possible partitions  
of  $N$  nodes



# Hierarchical Clustering

# Hierarchical Clustering

1. Build a similarity matrix for the network
2. *Similarity matrix*: how similar two nodes are to each other → we need to determine from the adjacency matrix
3. Hierarchical clustering iteratively identifies groups of nodes with high similarity, following one of two distinct strategies:
  - Agglomerative algorithms* merge nodes and communities with high similarity.
  - Divisive algorithms* split communities by removing links that connect nodes with low similarity.
4. *Hierarchical tree* or *dendrogram*: visualize the history of the merging or splitting process the algorithm follows. Horizontal cuts of this tree offer various community partitions.



# Agglomerative Algorithms

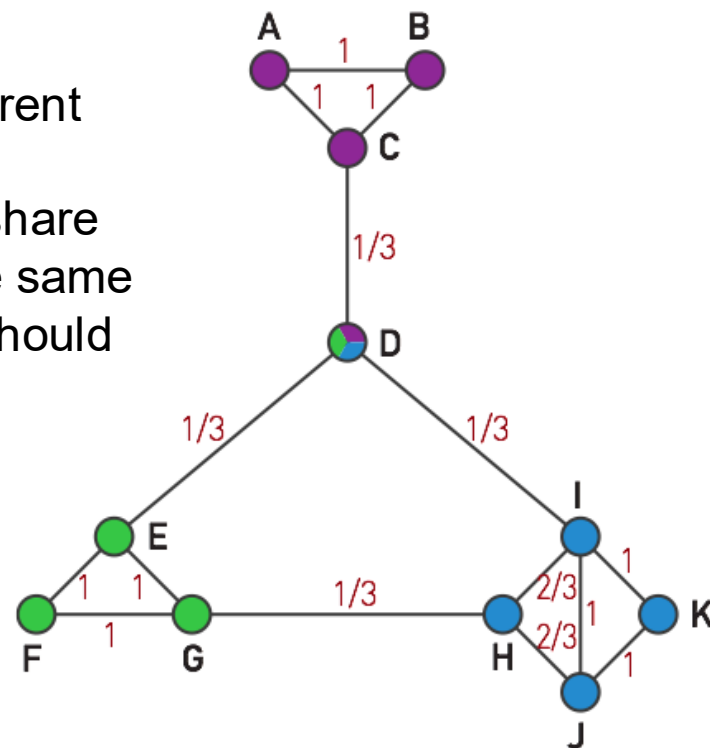
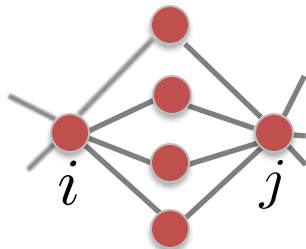
*Agglomerative algorithms* merge nodes and communities with high similarity.

## Step 1: Define the Similarity Matrix (Ravasz algorithm)

- High for node pairs that likely belong to the same community, low for those that likely belong to different communities.
- Nodes that connect directly to each other and/or share multiple neighbors are more likely to belong to the same dense local neighborhood, hence their similarity should be large.

Topological overlap matrix:  $x_{ij}^o = \frac{J_N(i, j)}{\min(k_i, k_j)}$

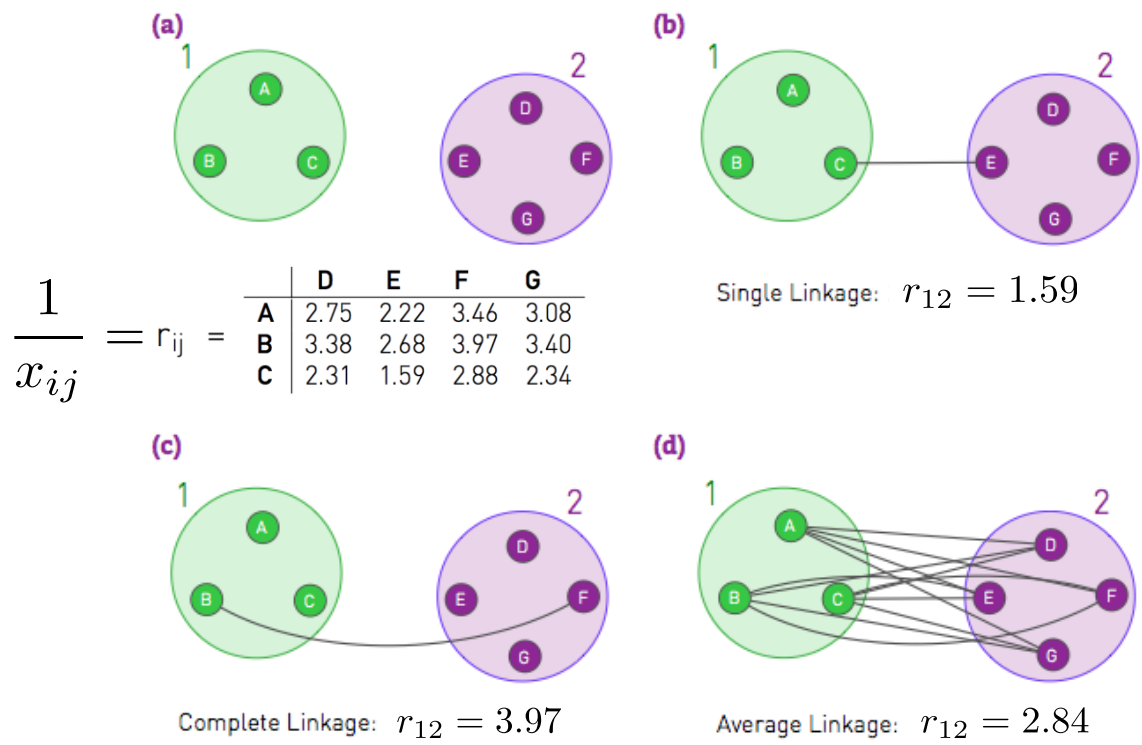
$J_N(i, j)$ : number of common neighbors of node  $i$  and  $j$ ;  
(+1) if there is a direct link between  $i$  and  $j$ ;



# Agglomerative Algorithms

## Step 2: Decide Group Similarity

- Groups are merged based on their mutual similarity through *single*, *complete* or *average cluster linkage*



E. Ravasz et al., *Science* 297 (2002).

A.-L. Barabási, *Network Science: Communities*.

## Step 3: Apply Hierarchical Clustering

- Assign each node to a community of its own and evaluate the similarity for all node pairs. The initial similarities between these “communities” are simply the node similarities.
- Find the community pair with the highest similarity and merge them to form a single community.
- Calculate the similarity between the new community and all other communities.
- Repeat from Step 2 until all nodes are merged into a single community.

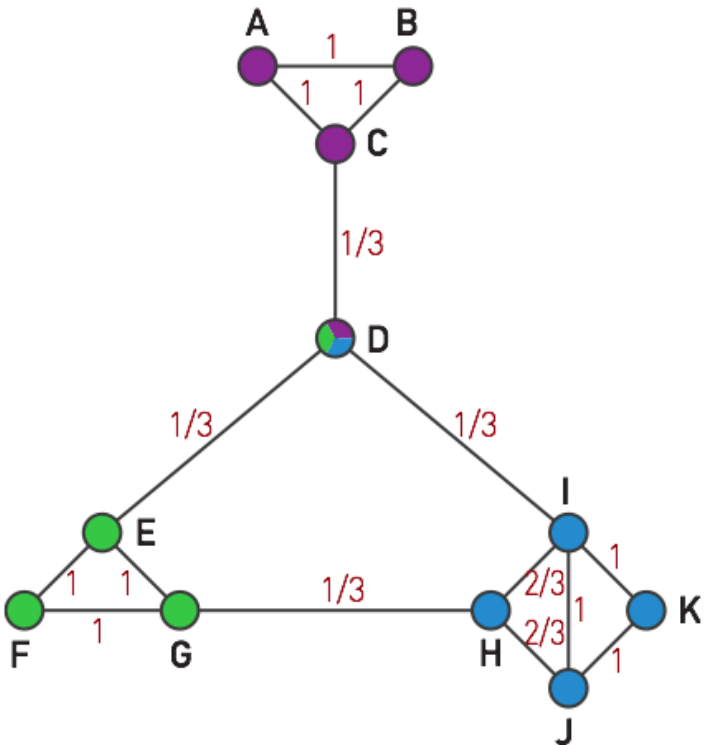
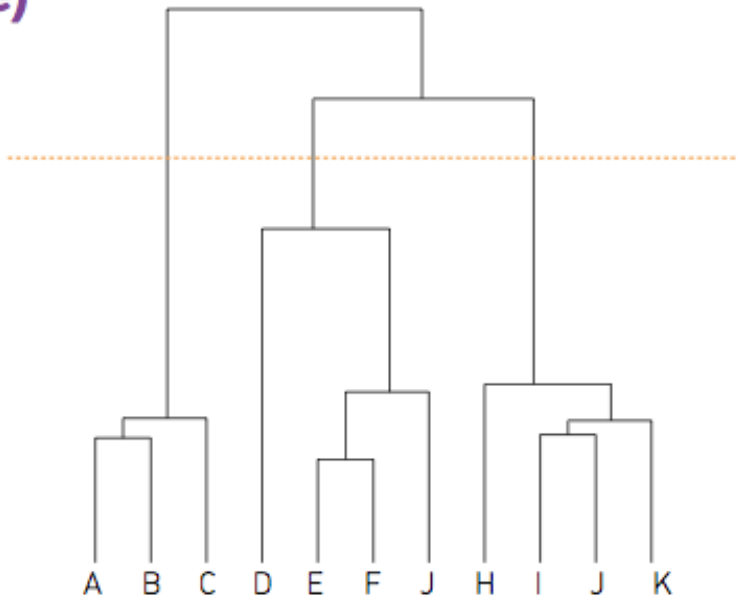
## Step 4: Build Dendrogram

- Describes the precise order in which the nodes are assigned to communities.

Step 4: Build Dendrogram

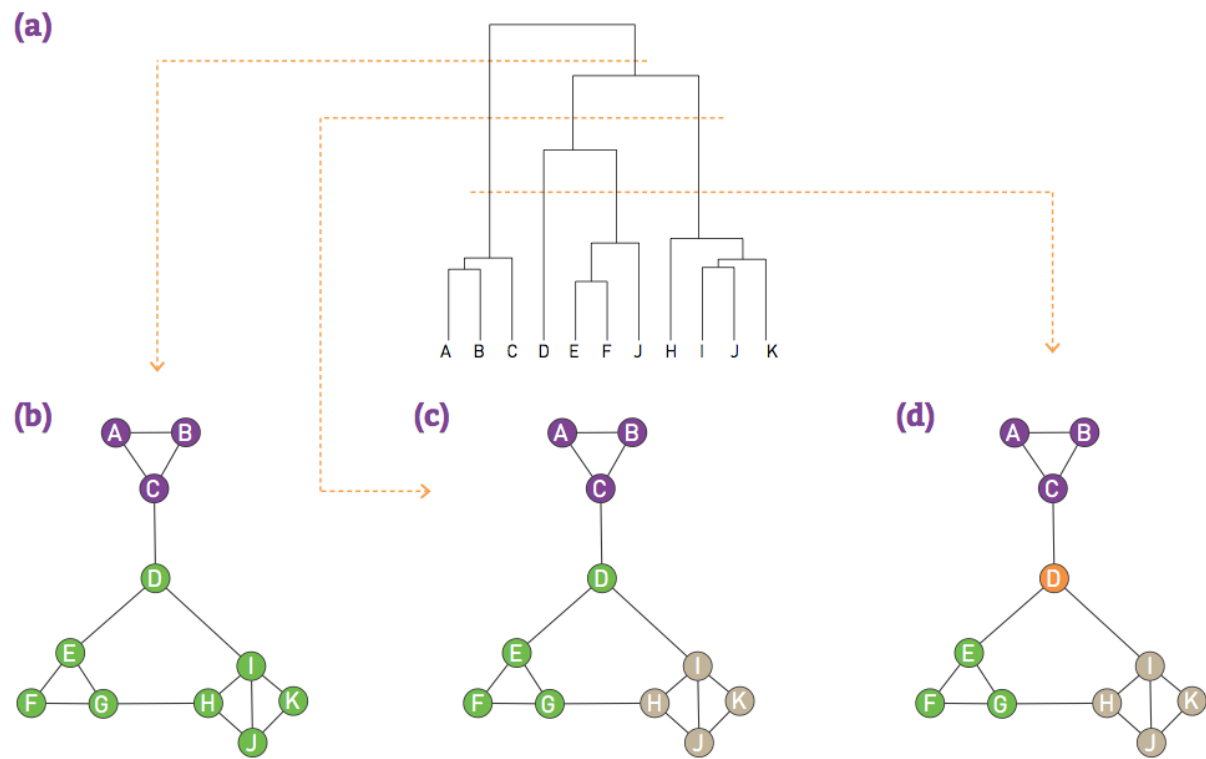
- Describes the precise order in which the nodes are assigned to communities.

(e)



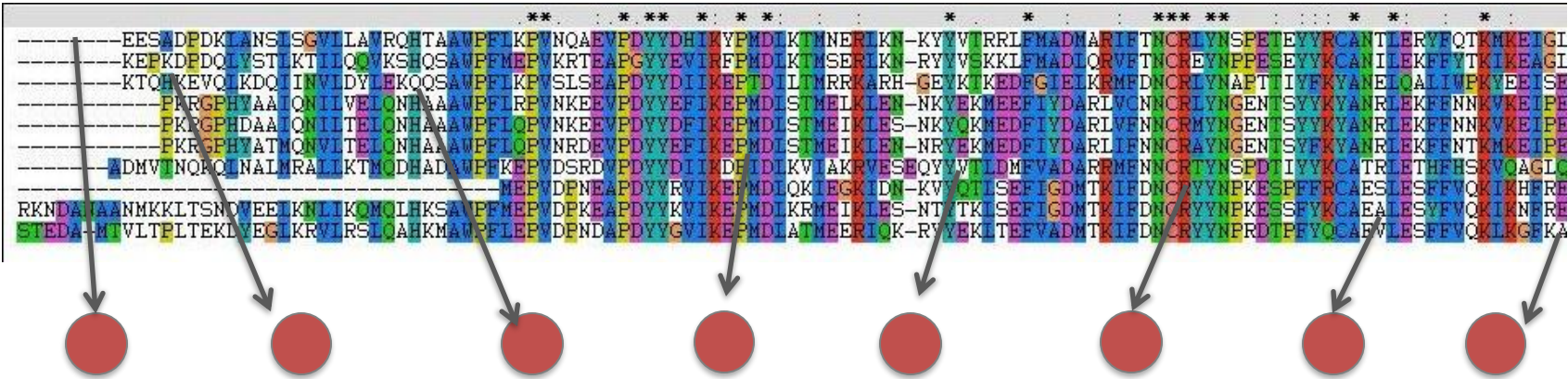
# Ambiguity in Hierarchical clustering

Where to “cut”?



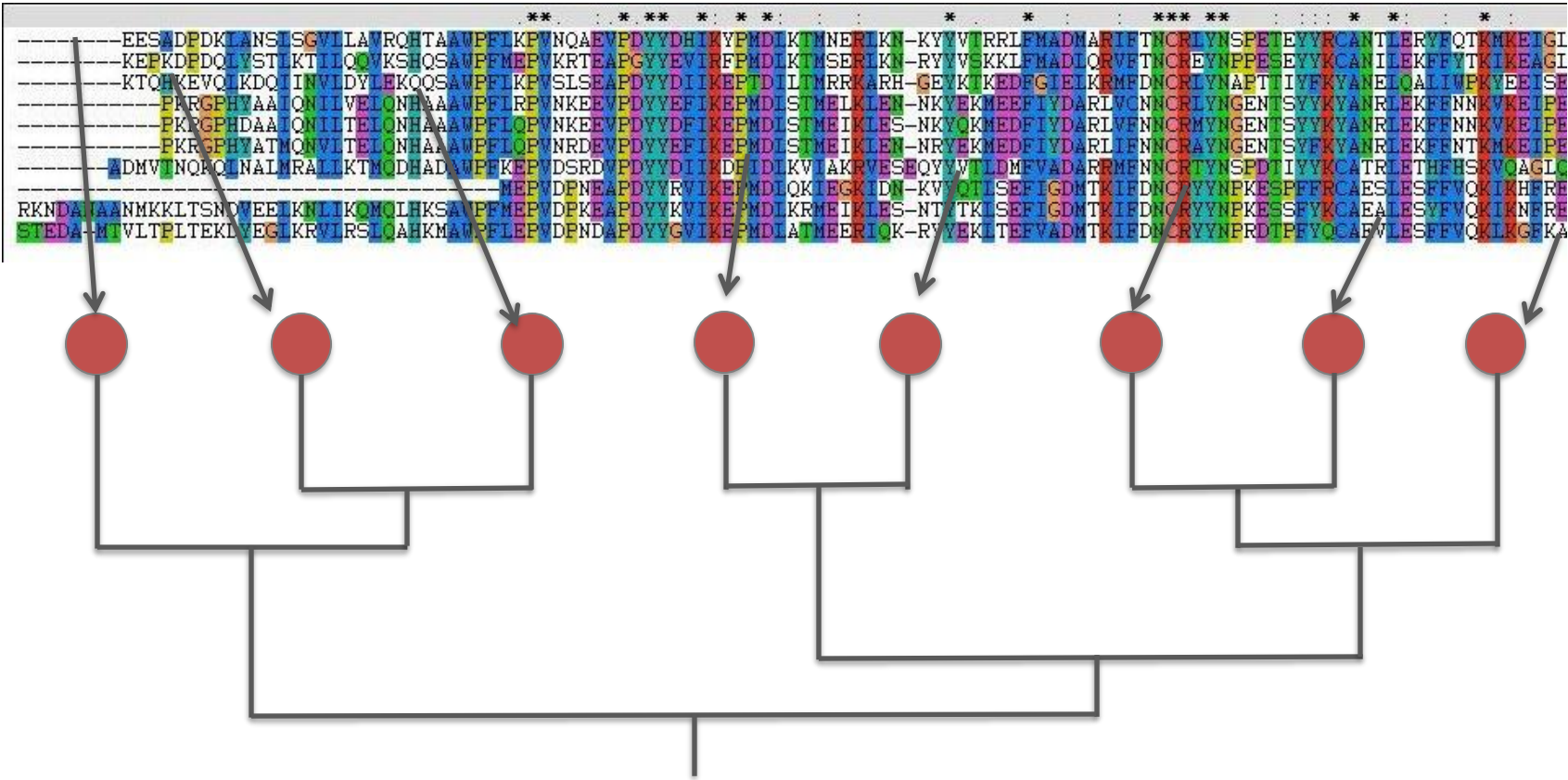
# Phylogenetic dendrograms

A similarity matrix is constructed between these sequences, by looking at how many aminoacids/nucleotides stay in place

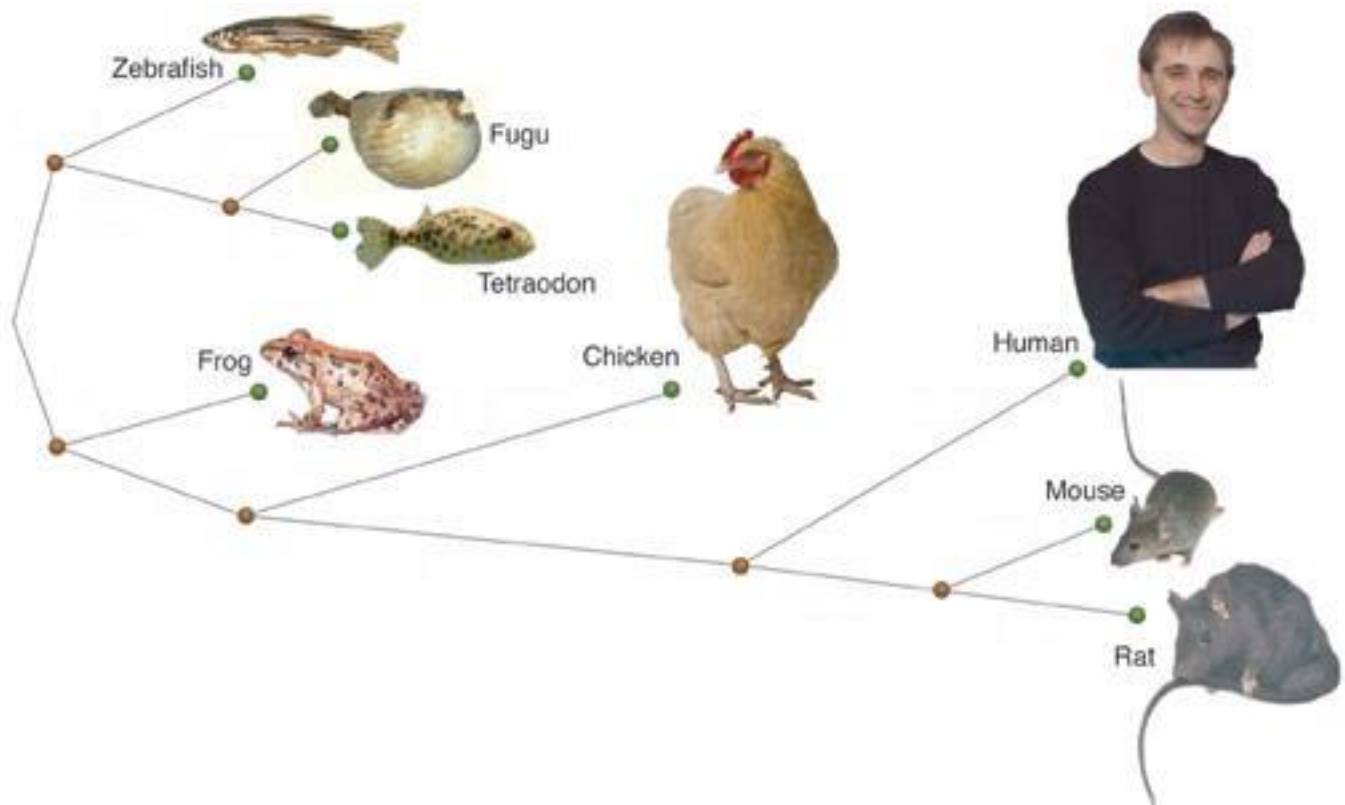


# Phylogenetic dendrograms

A similarity matrix is constructed between these sequences, by looking at how many aminoacids/nucleotides stay in place

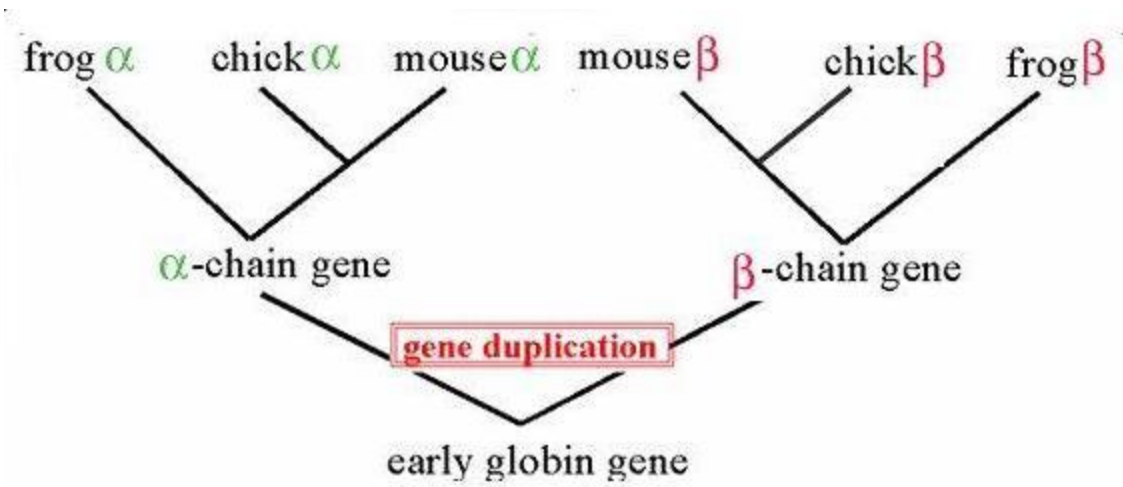


# Phylogenetic dendrograms





# Phylogenetic dendrograms



# Modularity

## H4: Random Hypothesis

Randomly wired networks are not expected to have a community structure.

## H4: Random Hypothesis

Randomly wired networks are not expected to have a community structure.

Imagine a partition in  $n_c$  communities  $\{C_c, c = 1, n_c\}$

Modularity

$$M(C_c) = \frac{1}{2L} \sum_{i,j=1}^N (A_{ij} - P_{ij}) \delta(C_i - C_j)$$

## H4: Random Hypothesis

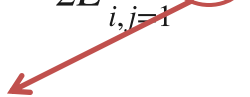
Randomly wired networks are not expected to have a community structure.

Imagine a partition in  $n_c$  communities  $\{C_c, c = 1, n_c\}$

Modularity

$$M(C_c) = \frac{1}{2L} \sum_{i,j=1}^N (A_{ij} - P_{ij}) \delta(C_i - C_j)$$

Original data

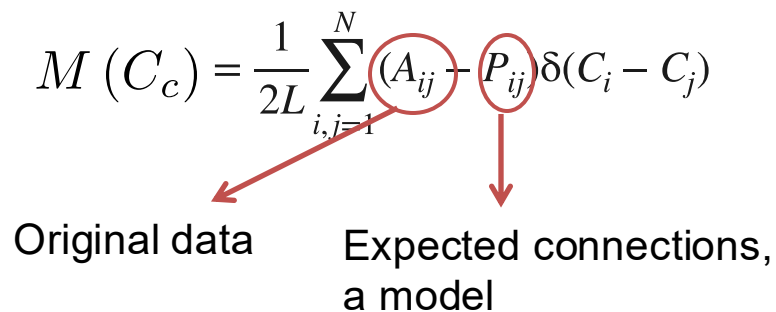


## H4: Random Hypothesis

Randomly wired networks are not expected to have a community structure.

Imagine a partition in  $n_c$  communities  $\{C_c, c = 1, n_c\}$

Modularity

$$M(C_c) = \frac{1}{2L} \sum_{i,j=1}^N (A_{ij} - P_{ij}) \delta(C_i - C_j)$$


Original data

Expected connections,  
a model

## H4: Random Hypothesis

Randomly wired networks are not expected to have a community structure.

Imagine a partition in  $n_c$  communities  $\{C_c, c = 1, n_c\}$

Modularity

$$M(C_c) = \frac{1}{2L} \sum_{i,j=1}^N (A_{ij} - P_{ij}) \delta(C_i - C_j)$$

Original data

Expected connections,  
a model

Relative to a specific  
partition

## H4: Random Hypothesis

Randomly wired networks are not expected to have a community structure.

Imagine a partition in  $n_c$  communities  $\{C_c, c = 1, n_c\}$

Modularity

$$M(C_c) = \frac{1}{2L} \sum_{i,j=1}^N (A_{ij} - P_{ij}) \delta(C_i - C_j)$$

Original data      Expected connections, a model      Relative to a specific partition

- Random network  $P_{ij} = 2L p_i p_j = \frac{k_i k_j}{2L}$
- Modularity is a measure associated to a partition



## Another way of writing $M$

$$M(C_c) = \frac{1}{2L} \sum_{i,j=1}^N (A_{ij} - P_{ij}) \delta(C_i - C_j) \quad P_{ij} = 2L p_i p_j = \frac{k_i k_j}{2L}$$

We can rewrite the first term as

$$\frac{1}{2L} \sum_{i,j=1}^N A_{ij} \delta(C_i - C_j) = \sum_{c=1}^{n_c} \frac{1}{2L} \sum_{i,j \in C_c} A_{ij} = \sum_{c=1}^{n_c} \frac{l_c}{L}$$

where  $L_C$  is the number of links within  $C$ . In a similar fashion, the second term becomes

$$\frac{1}{2L} \sum_{i,j} \frac{k_i k_j}{2L} \delta(C_i - C_j) = \sum_{c=1}^{n_c} \frac{1}{(2L)^2} \sum_{i,j \in C_c} k_i k_j = \sum_{c=1}^{n_c} \frac{k_c^2}{4L^2}$$

Finally we get:

$$M(C_c) = \sum_{c=1}^{n_c} \left[ \frac{l_c}{L} - \left( \frac{k_c}{2L} \right)^2 \right]$$

## H5: Maximal Modularity Hypothesis

The partition with the maximum modularity  $M$  for a given network offers the optimal community structure

## H5: Maximal Modularity Hypothesis

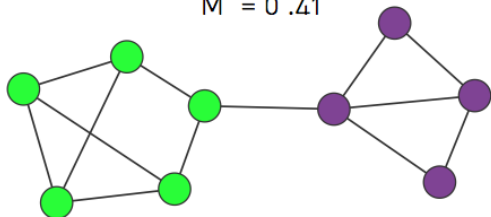
The partition with the maximum modularity  $M$  for a given network offers the optimal community structure

### Goal

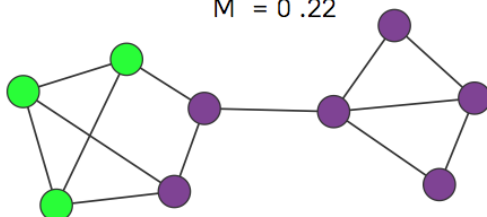
Find  $\{C_c, c = 1, n_c\}$  that maximizes  $M$

Which partition  $\{C_c, c = 1, n_c\}$  ?

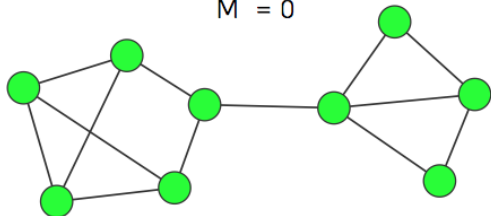
(a) OPTIMAL PARTITION  
 $M = 0.41$



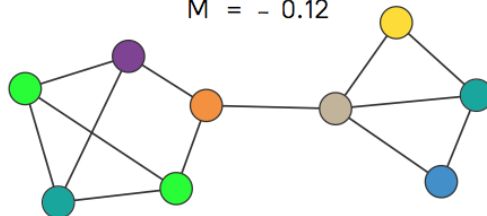
(b) SUBOPTIMAL PARTITION  
 $M = 0.22$



(c) SINGLE COMMUNITY  
 $M = 0$



(d) NEGATIVE MODULARITY  
 $M = -0.12$



- *Optimal partition*, that maximizes the modularity.
- *Sub-optimal* but positive modularity.
- *Negative Modularity*: If we assign each node to a different community.
- *Zero modularity*: Assigning all nodes to the same community, independent of the network structure.
- *Modularity is size dependent*

# Modularity based community identification

A *greedy algorithm*, which iteratively joins nodes if the move increases the new partition's modularity.

**Step 1.** Assign each node to a community of its own. Hence we start with  $N$  communities.

**Step 2.** Inspect each pair of communities connected by at least one link and compute the modularity variation obtained if we merge these two communities.

**Step 3.** Identify the community pairs for which  $\Delta M$  is the largest and merge them. Note that modularity of a particular partition is always calculated from the full topology of the network.

**Step 4.** Repeat step 2 until all nodes are merged into a single community.

**Step 5.** Record for each step and select the partition for which the modularity is maximal.

**Which partition  $\{C_c, c = 1, n_c\}$  ?**

- It can be used to design new algorithms, **aiming at maximizing  $M$**
- Modularity can be used to compare different partitions provided by other algorithms, like hierarchical clustering

The greedy algorithm is neither particularly fast nor particularly successful at maximizing  $M$ .

*Scalability:* Due to the sparsity of the adjacency matrix, the update of the matrix involves a large number of useless operations. The use of data structures for sparse matrices can decrease the complexity of the computational algorithm to , which allows us to analyze is of networks up to nodes. See

**"Fast Modularity" Community Structure Inference Algorithm**

<http://cs.unm.edu/~aaron/research/fastmodularity.htm> for the code.

A fast greedy algorithm was proposed by Blondel and collaborators, that can process networks with millions of nodes. For the description of the algorithm see

**Louvain method: Finding communities in large networks**

<https://sites.google.com/site/findcommunities/> for the code.